Embedded YOLO from scratch

Status, current results and the next training and test campaigns

Léonard Rivals

22 October 2026

In three points

Where we are
The PC side of the chain is complete and bit-exact for four networks. Safety evidence (equivalence, ABFT, latency bound) is in place. The board is not available yet.

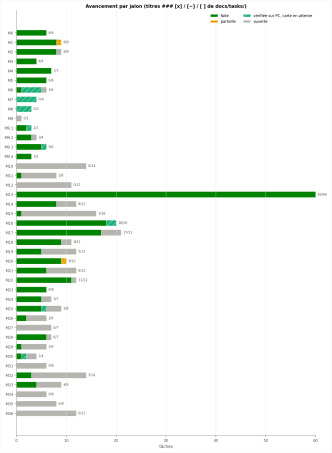
What we measured
VOC: 56.30 float $\rightarrow$ 55.66 INT8 $\rightarrow$ $55.66^{\text{C-sim}}$. COCO v4-tiny: 41.4 $\rightarrow$ 40.1 AP50. KV260: $37.2^{\text{proj.}}$ ms per image.

What is left to run
$\approx$ 40 h of PC CPU, $\approx$ 12 h of C-sim and $\approx$ 110–120 h of Colab GPU. Four transfers unblock most of it.

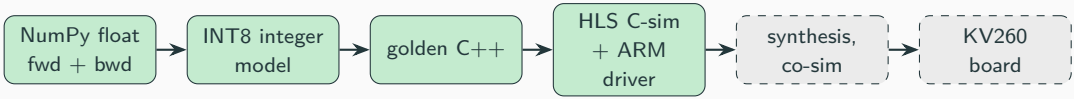
Where we are

Progress by milestone

Milestone	Tasks	Done	Progress
M0	6	6	100 %
M1	9	8	89 %
M2	9	8	89 %
M3	4	4	100 %
M4	7	7	100 %
M5	6	6	100 %
M6	6	1	17 %
M7	4	0	0 %
M8	3	0	0 %
M9	1	0	0 %
M9.1	3	2	67 %
M9.2	4	3	75 %
M9.3	6	5	83 %
M9.4	3	3	100 %
M10	14	0	0 %
M12	11	0	0 %
M11	8	1	12 %
M13	60	60	100 %
M14	12	8	67 %
M15	16	1	6 %
M16	20	18	90 %
M17	21	17	81 %
M18	11	9	82 %
M19	12	5	42 %
M20	10	9	90 %
M21	12	6	50 %
M22	12	11	92 %
M23	6	6	100 %
M24	7	5	71 %
M25	9	5	56 %
M26	6	2	33 %
M27	7	0	0 %
M28	7	6	86 %
M29	6	1	17 %
M30	4	1	25 %
M31	6	0	0 %
M32	14	3	21 %
M33	9	4	44 %
M34	6	0	0 %
M35	8	0	0 %
M36	12	0	0 %
Total	**397**	**231**	**58 %**



The chain today



Network	PC chain	Next
Tiny-YOLOv2 VOC	float, int, golden, C-sim: identical on 4,952 images	board
Tiny-YOLOv3 COCO	identical up to C-sim (20 images)	full COCO C-sim pass
YOLOv4-tiny COCO	identical up to C-sim (20 images, smoke run 10)	full C-sim, QAT
YOLOv7-tiny COCO	exported, calibrated; 34.3 AP at 416 (paper 35.2)	C-sim pass, VOC training
YOLOv5n COCO	torch-free, 28.1 AP at 640 (published 28.0); trainable	VOC training, integer
YOLOv8n COCO	torch-free reader	mAP, training (M35)

done on the PC needs Vitis / board, parked in `docs/tasks/deferred/`

source: `results/equivalence.md`, `results/map_stades.md`, `docs/tasks/M33-yolov5n.md`

Since the last deck (20 October)

Finished

- **M28 equivalence**: matrix per layer and stage, exhaustive operator tests, z3 proof of the scalar kernels, traceability checked in CI, tool-chain risks, post-processing properties (6 / 7, board row left)
- **M30**: static latency bound of the accelerator (T30.0)
- **M32**: Oxford-IIIT Pet boxed from the trimaps, datasheets
- **M33**: `.pt` read without torch, network from the Ultralytics YAML, C3 / SPPF in forward and backward; v5 decoding (28.1 AP), targets, CIoU loss, training recipe (T33.0–T33.7)
- **M34**: YOLOv7-tiny trainable without

Planned

- **M33–M35**: VOC fine-tuning of YOLOv5n and YOLOv7-tiny (6 and 10 CPU days), YOLOv8n (task-aligned assigner, DFL), integer models
- **M36**: decision explanation at the three stages: gradient, perturbation and exact integer attributions, miss causes, on-chip CAM
- the run inventory `long-runs.md`: every open long run with its command, its prerequisites and a duration

Open work per active milestone

Milestone	Done	Blocked by	Milestone	Done	Blocked by
M15 training campaign	1/16	GPU hours	M29 runtime monitors	1/6	T27 outputs
M24 YOLOv4-tiny	5/7	C-sim night, GPU	M30 latency bound	1/4	board
M25 dependability	5/9	CPU nights	M31 assurance case	0/6	M27, M29
M26 recent models	2/6	C-sim, licence	M32 TrashNet, Pet	3/14	labelling, Drive
M27 safety metrics	0/7	CrowdHuman on PC	M33 YOLOv5n	6/9	CPU days, torch fixture
M28 equivalence	6/7	board	M34 YOLOv7-tiny	2/6	CPU days, AP gap
			M35–M36	0/20	code

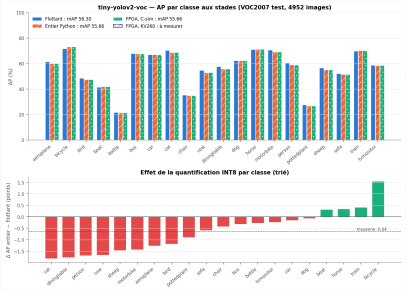
Most open tasks wait for **compute**, not for code: the runs are written, only the hours are missing. Everything that needs Vitis, Vivado or the KV260 is set aside in `docs/tasks/deferred/`, with the order to pick it up.

source: `docs/tasks/M*.md` (ticked tasks, 2026-10-23)

Current results

mAP at the three stages (VOC2007 test)

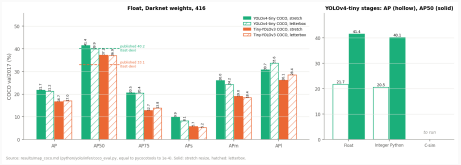
Stage	mAP	Equality with the integer model	Status
Float (fused BN, NumPy float32)	56.30	—	measured
Integer, Python (`IntNetwork`, bit-exact)	55.66	reference	measured
FPGA, C-sim (ARM driver, sim backend, PC)	55.66	4,952 / 4,952 identical images	csim
FPGA, KV260 (uio backend)	—	to be measured	—



On the **4,952 images**, the C-simulated kernel behind the ARM driver gives detections **identical** to the Python integer model: the whole loss (-0.64) is quantization.

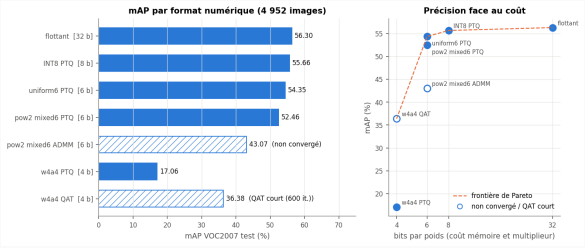
COCO val2017: three networks

Network	Preprocessing	AP	AP50	AP75	APs	APm	APl
YOLOv4-tiny	stretch	21.7	41.4	20.5	9.9	26.0	30.7
YOLOv4-tiny	letterbox	21.2	39.9	20.4	8.1	24.2	33.6
Tiny-YOLOv3	stretch	16.7	37.2	12.7	5.7	19.0	26.1
Tiny-YOLOv3	letterbox	17.0	36.9	13.8	5.2	18.4	28.4



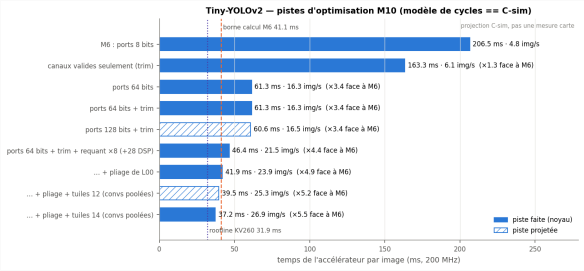
YOLOv4-tiny: 41.4 AP50 float (published $40.2^{\text{pub.}}$), 40.1 in INT8 (-1.3) $\rightarrow$ a QAT run is on the list.

Number formats on the same model



- INT8: 55.66; integer post-processing 55.56
- 6 bits uniform 54.35, mixed 52.46
- powers of two (ADMM) 43.07: **not converged**
- 4 bits: PTQ 17.06, short QAT 36.38 $\rightarrow$ **long QAT to run**

Projected performance on the KV260 (projection, not measured)



source: `results/latency_bound.md`

- M6 kernel $206.5^{\text{proj.}}$ ms $\rightarrow$ **$37.2^{\text{proj.}}$ ms** after M10 ($26.9^{\text{proj.}}$ img/s)
- two MACs per DSP: $24.7^{\text{proj.}}$ ms
- v3 $36.1^{\text{proj.}}$ ms, v4-tiny $51.8^{\text{proj.}}$ ms
- static worst-case bound (M30), v2: accelerator 83.4 ms, whole frame with post-processing 92 ms

Cycle model = C-sim counters; no board number yet.

Dependability and equivalence evidence

Equivalence (M28), four exports

- integer → golden → C-sim: **pointwise identical**, every byte
- fake-quant → integer: ≤ 1 LSB per layer (bounded proximity)
- scalar kernels proved with z3; exhaustive operator tests
- co-sim and board rows: pending (Vitis, KV260)

Faults (M25)

- exact int32 ABFT in Python, golden and the HLS kernel: 0 false alarm on 103 layers and 158 outputs
- sensitivity campaign: **smoke-run only**

Out-of-distribution monitor

- plain feature norm of the last backbone layer, read on the ARM
- AUROC 0.74 (low light), **0.88** (thermal), for a negligible share of the MACs
- beats Mahalanobis and an autoencoder

Latency bound (M30)

- the accelerator schedule does not depend on the image: static bound from the manifest alone
- post-processing is the data-dependent part, bounded by its 256 slots

source: `results/equivalence.md`, `results/faults.md`, `results/ood_monitor.md`, `results/latency_bound.md`

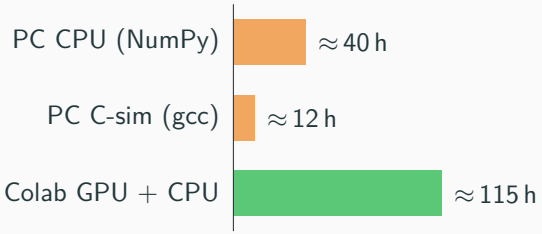
Beyond VOC: first sweeps (tier R)

Dataset	Training images	Best run	Batch	Training subset	Metric	Score (50 images)	Status
VOC	16,551	`b32-sall`	32	all	mAP, 11 points	36.27	tier-R
VisDrone	6,471	`b32-sall`	32	all	mAP, 11 points	2.24	tier-R
KITTI	5,985	`b32-sall`	32	all	mAP, 11 points	2.38	tier-R
FLIR	10,742	`b8-sall`	8	all	AP@[.5:.95] (COCO)	0.70	tier-R
ExDark	3,000	`b32-sall`	32	all	mAP, 11 points	13.29	tier-R
CrowdHuman	15,000	`b16-sall`	16	all	mAP, 11 points	35.87	tier-R

600 iterations, scored on 50 images: these numbers **rank** runs, they are not results (VOC v2 gets 63.51 on these 50 images against 56.30 on the full split). No fine-tuned run has converged yet: this is what the training campaign has to fix.

Next campaigns

The budget (estimates)



- PC CPU: ≈ 40 h + 2 nights (QAT grid 9 h, faults 10 h, ADMM 4 h)
- C-sim: shares the CPU, one at a time, at night
- Colab: YOLOv4-tiny on 11 datasets ≈ 33 h, drones 14 h, VisDrone sizes 11 h
- Colab ranges 110–120 h; unmeasured runs not counted

Reference speeds: T4, 6,000 it at batch 32 ≈ 1 h 45; VOC test eval on the PC ≈ 15 min float; C-sim ≈ 0.5 s per image.

source: `docs/tasks/long-runs.md` (2026-10-20)

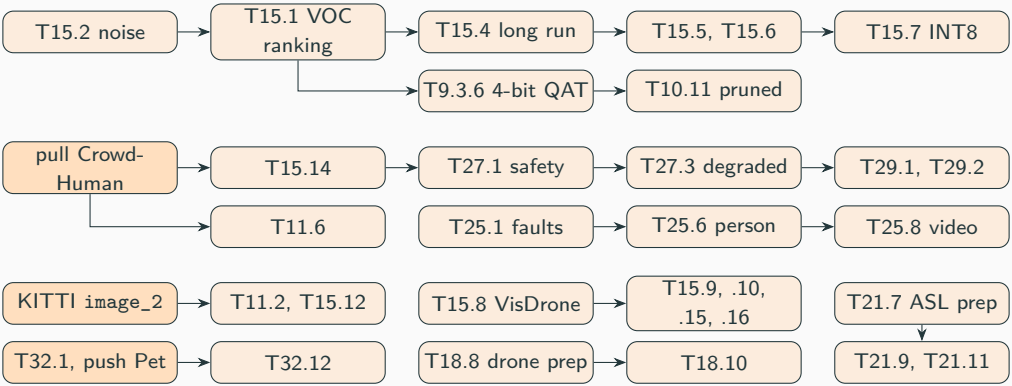
Unblock first: data transfers and labels

What	Unblocks	Size	Estimate
pull CrowdHuman to the PC	T11.6, T15.14, T27 safety metrics, T25.6, M29	11 GB	unmeasured
finish KITTI `image_2`	T11.2 out-of-domain, T15.12 KITTI training, T16.14	≈ 12 GB	unmeasured
push Pet, then TrashNet to Drive	T32.12 fine-tuning on Colab	798 MB	unmeasured
T32.1 TrashNet labels (Grounding DINO)	T32.10 stats, T32.12	2,427 images	1.5–2 h
FLIR to Drive (from Colab)	T15.13 FLIR training	12 GB	unmeasured
T18.8 drone datasets prep + stats	T18.10 drone training	1.2 M objects	M

Transfers over 10 GB are slow and sometimes freeze: start them first, alongside the first night runs.

source: `docs/tasks/long-runs.md`, Transfers

Dependencies between the runs



unblocker first; the rest follows the arrows.

source: docs/tasks/long-runs.md, Totals and order

Training campaigns (Colab GPU) (estimates)

Task	Run	Estimate
T15.2 → T15.1	VOC: noise of a run (2 seeds), then full-split ranking of the sweep	40 min + 2 h 15–3 h 40
T15.4	VOC long run: batch 32, 6,000 it, LR steps 4,800 / 5,400 (extend to 20,000)	1 h 45 (+ 4 h)
T15.5, T15.6, T15.7	LR × burn-in axes, batch 64, then INT8 of the selected run	1 h + 3 h 30 + 30 min
T15.8 → T15.9	VisDrone long run at 416, then 608 and 832 inputs	1 h 45, then 3 h 40 + 7 h
T15.10, .15, .16, .11	VisDrone: ignored regions, letterbox, tiles 640, one head vs two	≈ 4 h
T15.12	KITTI: long runs 416 letterbox and 640 × 192, two sweeps	≈ 5 h
T15.13	FLIR: 3-channel sweep, long runs 1 and 3 channels	≈ 5 h
T15.14	ExDark, CrowdHuman: full-split mAP, person recall	2 h CPU
T9.3.6, T10.11	4-bit QAT (8→6→4 ramp), long fine-tune of pruned r30 / r50 / r70	5 h + 6 h
T24.6	YOLOv4-tiny sweep + long run + INT8 on the 11 trainable datasets	30–35 h
T18.10, T21.9, T32.12	drones (4 sets), ASL, TrashNet + Pet	14 h + 3 h 30 + 6 h

source: docs/tasks/long-runs.md, Colab

Test and evaluation campaigns (PC nights) (estimates)

Task	Run	Estimate
T12.2–T12.4	M12 profiles: mAP subsets, fake-quant, PTQ variants on the full split	2 h + 20 min + 50 min
T12.5	QAT grid: ref, LR $3 \cdot 10^{-4}$ / 10^{-3}, 300 / 2,000 it, batch 16, no steps	8–10 h (2 nights)
T12.6	ADMM powers of two, 6 variants (REQ-YOLO)	4 h
T12.7, T12.9	HW post-processing (+ 100k random heads), C-sim 100 / 500 images	30 min + 1 night
T25.1	fault sensitivity: BER, layer and bit grids, 3 seeds, VOC 500	5.4 h
T25.6	critical errors for a person detector (CrowdHuman)	5 h
T27.1, T27.3	person safety metrics at fixed FPPI; 6 degradations × 3 severities	1 h + 1.5–7 h
T11.1, T11.2, T11.6	COCO integer; out-of-domain (ExDark, KITTI, VisDrone); CrowdHuman	30 min + 1.5 h + 45 min
T32.6, T32.7	Pet out-of-domain; M22 labeller against the trimap boxes	20 min + 20 min
T13.30, T12.11	whole test suite with durations; GPU tests on the RTX 1000 Ada	1 night; driver first

source: `docs/tasks/long-runs.md`, PC CPU

C-sim campaigns (gcc, at night) (estimates)

Task	Run	Estimate
T12.9	`make bench-sim`: Tiny-YOLOv2, VOC 4,952	40 min
T11.1	Tiny-YOLOv3 COCO 5,000 (after `coco-int`)	45 min
T24.5	YOLOv4-tiny COCO 5,000, three-stage mAP	1 h 30
T26.0, T26.3, T26.4	YOLOv7-tiny, YOLOv5n, YOLOv8n COCO	1 h 30 + 3 h
T27.1	person metrics at the C-sim stage, CrowdHuman val (a subset is fine)	2–3 h
T30.1	cycle distribution over $\geq$ 10,000 real images	1 h 30
T11.6	CrowdHuman cycles, 200 images	5 min

Each pass must keep **C-sim detections = integer detections**: a single difference stops the campaign.

source: `docs/tasks/long-runs.md`, PC C-sim

Proposed schedule (estimates)

Week	PC CPU (day, night)	PC C-sim (night)	Colab
1	transfers (CrowdHuman, KITTI, Pet); T32.1 labels; T25.1 faults	`bench-sim`, T11.1 COCO v3	T15.2, T15.1 VOC; T15.8 VisDrone
2	T12.5 QAT grid (2 nights); T11.6, T15.14	T24.5 YOLOv4-tiny	T15.4 VOC long run; T15.9 608 / 832
3	T12.6 ADMM; T27.1, T27.3 safety; T25.6	T27.1 C-sim; T30.1	T15.5–T15.7; T15.12 KITTI; T9.3.6 4-bit
4	T12.2–T12.4, T12.7; T29.1, T29.2 monitors	T26.x once M33–M35 give mAP	T24.6 YOLOv4-tiny, T32.12, T18.10 (spread over 2–3 weeks)

Rule: one heavy job per machine at a time; each result goes to the task's **Done** field and its row leaves `long-runs.md`; tier R ranks, only full splits are published.

Risks and decisions

Risks and decisions

- **No board measurement yet**: latency is a projection, the FPGA mAP is a C simulation. Synthesis, co-sim and board runs wait in `deferred/`
- **Data on the PC**: CrowdHuman is on Drive only; KITTI is incomplete. They block the safety chain (M27 $\rightarrow$ M29 $\rightarrow$ M31)
- **PC GPU**: an RTX 1000 Ada is seen, but no driver: install it, or keep all training on Colab
- **CPU training speed**: logs give 0.7 s per image, the docs still say 0.45. The night estimates use 0.7
- **Licence**: YOLOv5n / YOLOv8n weights are AGPL-3.0. Check it before storing them on Drive or quoting numbers in the article
- **Convergence**: ADMM and 4-bit QAT are not converged yet. The long runs decide whether these formats stay in the article

source: `docs/tasks/long-runs.md`, Inconsistencies

Summary

- The PC chain is **bit-exact** for four networks: 56.30 $\rightarrow$ 55.66 $\rightarrow$ $55.66^{\text{C-sim}}$ on VOC
- The equivalence evidence is complete on the PC. ABFT and the latency bound are in place
- What is missing is **compute hours**, not code: $\approx$ 40 h CPU, 12 h C-sim, 110–120 h GPU
- Week 1: start the transfers and the first VOC and VisDrone runs. The training campaign turns the tier-R rankings into full-split results
- In parallel: M33–M35 (recent models without PyTorch), M36 (explanation)

Questions?

Where to find the plots

What	Link
Gallery of every figure	`results/figures.md`
mAP, formats, calibration, QAT / ADMM curves	`results/figures/resultats/`
Progress, timeline, tests	`results/figures/projet/`
Metric tables	`results/` (`map_*.md`, `faults.md`, `equivalence.md`)
Sweeps: loss curves, rankings, AP per class	`docs/tasks/figures/resultats/`, `analyse_balayages.ipynb`
Executed notebooks per dataset	`notebooks/`
Open long runs	`docs/tasks/long-runs.md`
Colab runs (local, `make harvest`)	`build/notebooks/<dataset>/<model>/runs/<run>/`
PC training runs (local)	`build/train/<run>/loss.csv`

Same list in the `README`.